

江南大学

《程序设计课程设计》
大作业

人工智能与计算机 学院 计算机科学与技术 专业

组 名: 黄炯鸡

组长姓名: 陈俊宇

组长学号: 1030425330

2026 年 6 月

一、项目简介

本项目是一款基于 C++20 开发的 2D 校园 RPG 冒险游戏系统，灵感来源于《女神异闻录 4》(Persona 4)。玩家可在城镇与学校地图中自由移动，白天通过与 NPC 对话推进 Social Link 社群羁绊，夜晚进入回合制战斗击败阴影敌人。系统涵盖角色成长、人格面具 (Persona) 装备与切换、背包管理、商店交易、任务追踪、等级成长以及存档读档等完整的 RPG 基础功能。

项目采用分层架构设计，核心领域模型 (Core) 为纯 C++ 实现，不依赖任何图形或数据库库，便于单元测试。上层通过抽象的引擎接口 (IRenderer / IWindow / IInput) 与 SFML 解耦，实现了良好的可测试性和可扩展性。数据持久化采用 SQLite3，构建系统采用 CMake，并支持跨平台编译 (Windows MinGW / MSVC、Linux GCC、macOS Clang)。项目使用 Git 进行版本控制，GitHub Actions 实现自动化持续集成。

GitHub 仓库地址: <https://github.com/nick-cjyx9/26-spring-cpp-group>

二、项目特色与创新

表 1 项目特色与创新

序号	附加功能	简介
一	数据库技术	使用 SQLite3 实现游戏数据的持久化存储，包括角色属性、Persona 数据、背包物品、Social Link 进度、任务状态等，支持多存档槽位读写。
三	图形用户界面技术	基于 SFML 2.6 实现 2D 顶视角图形渲染，包含标题画面、城镇/夜晚地图、战斗场景、商店、背包等十余个交互场景。
六	STL 高级应用	综合使用 <code>std::vector<std::unique_ptr<Item>></code> 管理背包与商店物品， <code>std::map<std::string, Quest></code> 管理任务， <code>std::map<std::string, SocialLink></code> 管理社群羁绊，以及 <code>std::vector<std::string></code> 记录战斗日志。
八	软件工程实践	采用 Git 分支模型协同开发，CMake Presets

		跨平台构建, GitHub Actions 自动化 CI, 轻量级纯 C++ 单元测试框架覆盖核心领域模型。
--	--	--

三、 项目管理

本节可以介绍团队分工、项目进度甘特图等。

表 2 项目特色与创新

序号	成员角色	姓名	学号	班级	具体分工
1	组长	陈俊宇	1030425330	计科 2503	项目架构设计、 GameManager 与核心战斗系统、Persona 机制、团队协作与代码审查
2	组员	王子衿	1030425331	计科 2503	数据持久化层 (DatabaseManager、SaveRepository)、SQLite3 数据库设计与实现
3	组员	路美杰	1030425329	计科 2503	场景系统开发 (TitleScene、TownScene、NightScene、BattleScene、UI 交互)
4	组员	祖程浩	1030425328	计科 2503	核心领域模型 (Character、Item、Inventory、Shop、Quest、SocialLink)、单元测试编写



图 1 项目进度计划甘特图

四、需求分析

4.1 功能性需求

角色与成长系统：玩家角色拥有等级、HP/SP、金币等属性，通过完成任务和击败敌人获取经验升级，升级后提升 HP/SP 上限。

Persona 人格面具系统：角色装备 Persona 获得力量（Strength）、魔力（Magic）、速度（Speed）三维战斗属性及元素抗性。游戏开始时从综合型初始 Persona 中随机抽取。战斗中可切换 Persona，Social Link Rank 提升可使当前 Persona 升级并学会新技能。

战斗系统：简化回合制战斗，战斗开始时按速度 roll initiative 生成固定行动队列。玩家可选择物理攻击、技能、切换 Persona、使用道具或逃跑。物理攻击可被闪避，技能消耗 SP 并受元素抗性影响（Weak / Resist / Null / Drain / Repel）。

Social Link 社群系统：早晨与特定 NPC 对话增加羁绊进度，Rank 提升后给予当前 Persona 等级成长和技能奖励。

商店与背包系统：商店出售恢复道具、装备和 Persona；背包使用 `std::vector` 管理物品，支持使用、装备和出售。

任务系统：支持接受、完成和领取奖励，条件支持击杀敌人数等。

存档读档系统：基于 SQLite3 实现多槽位存档，完整保存角色状态、Persona、背包、Social Link 和任务进度。

4.2 非功能性需求

可测试性：核心层（Core）不依赖 SFML 和 SQLite3，可通过轻量级测试框架在 headless 环境下运行单元测试。

跨平台性：CMake Presets 支持 Windows（MinGW/MSVC）、Linux（GCC）、macOS（Clang）多平台编译。

可扩展性：通过 IRenderer / IWindow / IInput 抽象接口解耦上层逻辑与具体渲染实现，便于替换底层图形库。

五、 系统设计

系统采用四层架构设计：

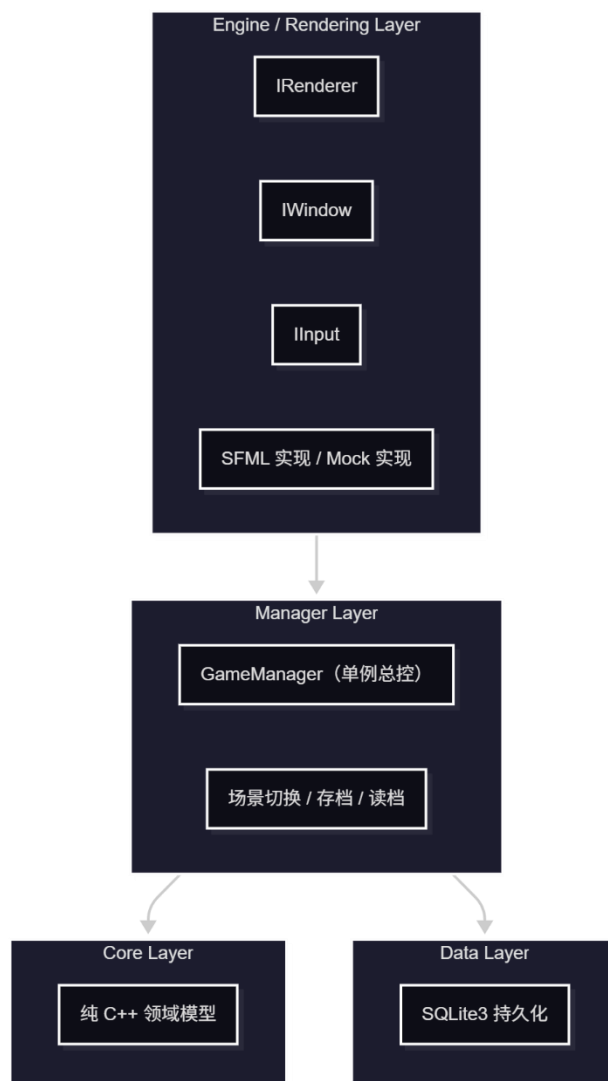


图 2 系统设计架构图

- **Engine 层**: 负责窗口管理、事件循环、输入处理和 2D 渲染。通过抽象接口与上层解耦。
- **Manager 层**: GameManager 单例持有全局游戏状态, 协调场景切换、保存与加载。
- **Core 层**: 纯 C++ 领域模型, 包含角色、Persona、物品、敌人、战斗、地图、任务、社群等全部业务逻辑。
- **Data 层**: DatabaseManager 负责 SQLite3 数据库连接与表管理, SaveRepository 负责具体的存档读写操作。

六、 关键功能实现

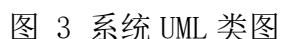
6.1 设计原则

项目严格遵循面向对象设计原则, 在核心领域模型中体现了封装、继承、多态与类之间的合理关联:

- **封装**: 所有类的字段均为 private/protected, 通过公共 getter/setter 暴露必要状态, 例如 Character::hp()、Persona::level()、SocialLink::rank() 等。
- **继承**: 抽象基类 Item、Enemy、Entity、IScene、IRenderer 等定义统一接口, 派生类实现具体行为, 例如 FoodItem、Slime、NpcEntity、BattleScene。
- **多态**: 通过虚函数实现运行时多态, 例如 item->use(character)、enemy->battleCry()、renderer->drawRect(...)、scene->update(...).
- **关联**: GameManager 组合 Character、Inventory、Shop、QuestManager、SocialLinkManager、TileMap, 作为全局状态协调中心。

6.2 系统 UML 类图

下图展示了 Campus RPG 的主要类及其关系。图中将类按四层架构分组, 清晰地体现了 Engine 接口层、Manager 层、Core 领域模型层与 Data 持久化层的职责边界。



Character: 玩家角色类，维护名称、等级、HP/SP、金币以及当前 Persona。战斗三维（attack / magic / speed）完全由当前 Persona 的基础属性与装备加成按等级倍率计算得出，角色自身不直接存储战斗属性。

`addPotentialSkill` 与 `checkSkillUnlocks` 支持按等级自动解锁潜在技能。等级成长只能通过 `Social Link Rank` 提升触发。

FoodItem / PotionItem 恢复 HP, SpItem 恢复 SP, EquipmentItem 提供装备加成, PersonaItem 表示可获得的 Persona。

BattleSystem: 战斗控制器，管理玩家与敌人实例。战斗开始
buildTurnOrder() 按速度 roll initiative 生成固定行动队列；玩家回合可执行
 攻击、技能、道具、切换 **Persona** 或逃跑，敌人回合执行其固定出招循环。

GameManager: 单例总控, 组合 Core 与 Data 层对象, 负责 newGame() 默认数据初始化、save() / load() 入口、场景切换与 NPC 池/日

期系统管理。

DatabaseManager / SaveRepository: DatabaseManager 管理 SQLite3 连接与表结构, SaveRepository 提供面向领域对象的高阶 saveAll() / loadAll() 接口, 将运行时状态序列化到五张核心表中。

七、 关键功能实现

7.1 开发环境搭建

项目使用 CMake 3.20+ 作为构建系统, SFML 和 SQLite3 通过 CMake FetchContent 自动下载, 无需手动安装。本地开发使用 VS Code + CMake Tools 扩展, 通过 CMake Preset 选择对应工具链 (windows-mingw-release 等)。单元测试通过 ctest 集成, GitHub Actions 在 Windows 上自动执行构建与测试。

7.2 Persona 属性计算与装备系统

Persona 是角色的战斗属性来源, 最终属性计算公式为:

$$\text{finalStat} = (\text{baseStat} + \text{equipmentBonus}) \times \text{levelMultiplier}$$

$$\text{levelMultiplier} = 1 + (\text{Persona 等级} - 1) \times 0.05$$

EquipmentItem 按槽位提供不同偏向的加成: Weapon 偏向力量、Armor 偏向魔力、Accessory 偏向速度、Relic 提供混合加成。装备的使用通过 Item::use(Character&) 多态接口实现, Character 在装备时更新自身加成系数, 并通过 attack() / magic() / speed() 方法实时计算最终属性值。

7.3 回合制战斗与 Initiative 队列

BattleSystem 在战斗开始时为所有参战单位计算 initiative:

$$\text{initiative} = \text{speed} \times (1 + \text{random}(-10\%, +10\%))$$

按 initiative 从高到低生成固定队列, 整局战斗按此队列循环。玩家回合通过 isPlayerTurn() 判断, 敌人回合执行其 attackPattern_ 中的下一个动作。物理攻击可被闪避, 闪避率基于双方速度差; 技能攻击不会被闪避, 但受目标元素抗性影响。

7.4 数据持久化

SQLite3 数据库包含 character、persona、inventory、social_link、quest_progress 五张表。DatabaseManager 负责初始化表结构, SaveRepository 提供 save() / load() 接口, 将 GameManager 中的运行时状态序列化到数据库。

加载时从数据库读取基础数据，Persona 的技能通过 ID 在 GameManager::initDefaultPersonas() 中重新绑定。

7.5 遇到的问题与解决方法

问题一： SFML 与 SQLite3 在不同机器上的安装路径不一致，导致 CMake 配置失败。

解决： 使用 FetchContent 在首次配置时自动下载依赖，并通过 CMakePresets.json 管理不同平台的预设配置，避免硬编码路径。

问题二： 核心层若直接依赖 SFML，会导致单元测试无法在 CI 无图形环境中运行。

解决： 将引擎抽象为 IRenderer / IWindow / IInput 接口，Core 层完全不依赖 SFML，测试目标仅链接 CampusRPGCore 库和 Mock 实现。

问题三： 加载静态贴图资源时报错 Cannot find ...

解决： 文件系统路径存在中文，被 SFML 解析成乱码，修改目录名以避免出现中文和空格即可

问题四： 多人协作时频繁修改同一热文件（如 Character.cpp、GameManager.cpp）导致合并冲突。

解决： 在 GitHub rule sets 中明确分支保护规则，要求关键文件变更必须通过 PR 并请求指定成员审查，避免直接推送至 main 分支。

八、 测试与结果分析

项目采用白盒测试与黑盒测试相结合的策略。Core 层使用纯 C++ 轻量级测试框架，无需外部库，可在 headless 环境下直接运行；LevelUp 与 Save 测试通过 Mock 渲染器与临时 SQLite 数据库验证场景渲染和数据持久化。测试使用自定义的 CHECK / CHECK_EQ 宏，运行后输出通过/失败总数：

```
E/d/m/2/c/b/w/tests on ↵main ≡ 1 MEM 47%
11:47:58 > .\CampusRPGTests.exe

==== CampusRPG core tests ====
run: 147 failed: 0
ALL PASSED
```

图 4 测试全通图

测试验证了角色成长、道具使用、装备加成、背包管理、商店交易、战斗流程、Social Link 进度与奖励、Persona 升级与技能解锁、实体碰撞、地图行

走性、升级场景渲染、SQLite 多槽位存取与旧版表结构迁移等核心功能。所有测试均通过，系统核心逻辑稳定可靠。

九、 总结展望

本项目完成了一个具有完整玩法循环的 2D 校园 RPG 系统，涵盖角色成长、Persona 装备与切换、回合制战斗、Social Link 社群、商店背包、任务追踪、存档读档等六大基础功能，并通过抽象引擎接口实现了上层逻辑与 SFML 的解耦。项目采用现代 C++ 特性（`std::unique_ptr`、`std::map`、`std::vector`）、CMake 跨平台构建、SQLite3 持久化以及 GitHub Actions 持续集成，体现了良好的软件工程实践。

未来可扩展方向包括：实现 Persona 合成系统（两个 Persona 融合生成新 Persona 并继承技能）；丰富任务条件（收集、等级等）；增加多存档槽位支持；扩展更多敌人类型与 Boss 战；以及引入剧情分支和更复杂的对话树系统。

十、 参考文献

[1] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002. [1] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2002.

[2] B. Meyer, “Applying ‘design by contract’,” *IEEE Trans. Softw. Eng.*, vol. 17, no. 10, pp. 1010–1021, Oct. 1992.

[3] 王珊, 萨师煊, 数据库系统概论, 第 5 版. 北京: 高等教育出版社, 2014.

[4] SFML Team, “SFML Documentation,” SFML. [Online]. Available: <https://www.sfml-dev.org/documentation/>. Accessed: May 2025.

[5] SQLite Consortium, “SQLite Documentation,” SQLite. [Online]. Available: <https://www.sqlite.org/docs.html>. Accessed: May 2025.

[6] GitHub, “26-spring-cpp-group,” [Online]. Available: <https://github.com/nick-cjyx9/26-spring-cpp-group>. Accessed: Jul. 2026.